

# OpenStack on Kubernetes(OpenStack-Helm)환경에서 QEMU 프로세스의 관측 방안

김준석<sup>1</sup>

<sup>1</sup>숭실대학교 소프트웨어학부 학부생

kimjoon0605@naver.com

## Monitoring and Observability Strategies for QEMU Processes in OpenStack on Kubernetes(OpenStack-Helm)

Joon-Seok Kim<sup>1</sup>

<sup>1</sup>Dept. of Software Science, Soongsil University

### 요 약

OpenStack-Helm(OSH)은 OpenStack을 쿠버네티스 파드(Pod)로 배포하여 관리 편의성을 제공하지만, QEMU 프로세스가 호스트의 machine cgroup 영역에 배치됨에 따라 kubelet, cAdvisor 등 쿠버네티스 표준 모니터링 도구가 VM의 자원 사용량과 자원 경합 상태를 관측하지 못하는 한계가 있다. 아울러 기존 OpenStack의 libvirt 기반 모니터링 역시 실시간 자원 경합 상태를 효과적으로 노출하지 못한다. 이를 해결하기 위해 본 연구는 libvirt 소켓과 호스트의 cgroup/procfs를 결합하여 자원 사용량과 커널 레벨 경합 지표(PSI)를 동시에 추출하는 읽기 전용 수집기(qemu-exporter)를 제안한다. 실측 실험 결과, 기존 쿠버네티스 표준 모니터링의 계측 누락 현상을 검증하였으며, 제안한 수집기가 노드 CPU 0.11% 수준의 낮은 오버헤드로 VM의 성능 및 자원 경합 상태를 정확히 관측함을 확인하였다.

### 1. 서론

쿠버네티스의 애플리케이션 관리 능력과 운영상 이점에 따라 전 세계적인 도입률이 지속적으로 증가하고 있다[1]. 이에 대응하여 오픈스택(OpenStack) 생태계에서는 OpenStack-Helm(OSH)을 활용하여 오픈스택 컴포넌트를 쿠버네티스 파드(Pod) 형태로 배포 및 관리하는 구조가 활성화되고 있다. OSH 환경에서 가상 머신(VM) 생성 시, Nova 컴포넌트는 nova-compute를 거쳐 libvirt 파드의 libvirtd를 통해 QEMU 프로세스를 실행한다. 그러나 생성된 QEMU 프로세스는 파드의 cgroup 영역(kubepods.slice)이 아닌 호스트 루트의 machine(혹은 machine.slice) cgroup에 배치된다.[2] 이로 인해 표준 쿠버네티스 모니터링 도구는 VM의 자원 사용을 인식하지 못하며, 기존 오픈스택의 libvirt API 기반 모니터링 역시 실시간 자원 경합 상태를 포착하지 못하는 한계가 존재한다. 이에 본 연구에서는 OSH, libvirt, Nova의 소스코드 수정 없이 실시간 자원 경합을 관측하는 읽기 전용(Read-Only) 수집기를 구현하고, 낮은 시스템 오버헤드를 검증함으로써 기존 모니터링 체계의 관측 공백을 해소하는 실용적 방안을 제시한다.

### 2. 이론적 배경

#### 2.1 OpenStack on Kubernetes(OpenStack\_Helm)

OpenStack on Kubernetes는 클라우드 인프라를 구축하는 오픈스택의 제어부(Control Plane) 및 주요 구성 요소를 쿠버네티스의 파드(Pod) 형태로 운영하는 아키텍처이다. 선언적 상태 관리 및 고가용성 보장 등 쿠버네티스의 핵심 기능을 활용함으로써, 오픈스택의 배포 및 서비스 관리 효율성을 향상시킬 수 있다. 이러한 배포 방식을 구현한 대표적인 프레임워크로 OpenStack-Helm(OSH)이 있다.

#### 2.2 Libvirt 통계 기반 모니터링(virsh domstats)

libvirt가 제공하는 통계 수치를 통해 VM의 CPU 사용 시간, 네트워크 트래픽, 스토리지 입출력(I/O) 용량 및 속도 등의 지표를 수집할 수 있다. 그러나 해당 통계는 단순 자원 사용량만을 나타내며, 자원 부족으로 인한 지연 시간인 자원 경합 지표는 제공하지 않는 한계가 있다.[3] libvirt는 경합 상태 확인을 위해 vcpu.N.wait 필드를 제공하지만, 이는 단편적인 누적 대기 시간일 뿐 커널의 PSI(Pressure Stall Information) 규격을 지원하지 않는다. 따라서 외부 모니터링 파이프라인과 직접 연동하여 경합 상태를 관측하기 어렵다.

#### 2.3 쿠버네티스 표준 모니터링(kubelet/cAdvisor)

쿠버네티스 환경에서 kubelet에 내장되어 모니터링을

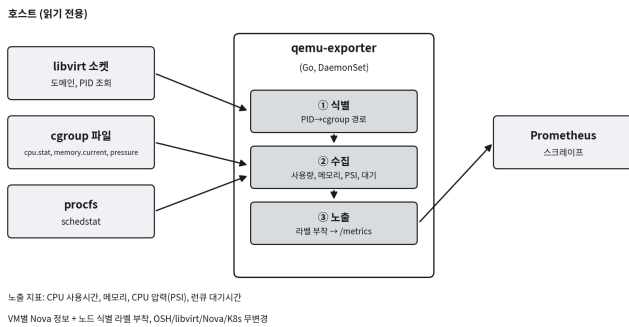
담당하는 cAdvisor는 kubepods cgroup 계층만을 수집 대상으로 삼으며, 호스트의 machine cgroup 계층은 탐색하지 않는다.[4] OpenStack-Helm 환경에서 QEMU 프로세스는 machine cgroup 하위에 배치되므로, cAdvisor 수집 대상에서 제외되어 VM 관련 모니터링 데이터가 근본적으로 존재하지 않는 관측 공백이 발생한다.

### 3. 연구 방법

#### 3.1 qemu-exporter 모니터링 에이전트

본 연구는 OpenStack-Helm(OSH), libvirt, Nova 및 쿠버네티스 소스코드의 수정 없이, 호스트의 libvirt 소켓과 cgroup/procfs 파일 시스템을 읽어 프로메테우스(Prometheus) 규격으로 노출하는 읽기 전용(Read-Only) 수집기 qemu-exporter를 제안한다. 에이전트의 주요 동작은 다음 세 단계로 구성된다.

<그림 1> qemu-exporter 구성도



첫째, “식별 단계”이다. libvirt 읽기 전용 소켓을 사용해서 동작 중인 VM의 PID를 알아내고, 호스트의 /proc/<PID>/cgroup 경로에서 도메인 스레드 탐색 알고리즘을 통해서 자원 계층을 위한 cgroup 경로를 찾는다. 둘째, “수집 단계”이다. 확보한 cgroup 경로에서 CPU 사용시간, 메모리 사용량(memory.current), CPU의 PSI 데이터(cpu.pressure)를 측정하고, procfs(schedstat)에서 스레드별 CPU 런큐 대기 시간을 측정한다. 셋째, “노출 단계”이다. 수집된 VM의 자원 사용량 및 자원 경쟁 지표에 Nova 메타데이터(인스턴스 ID, 이름, 인스턴스 타입 등)와 노드 식별 라벨을 결합하여 프로메테우스 서버가 스크레이프(Scrape)할 수 있는 표준 규격(/metrics)으로 노출한다. 이후 Prometheus는 /metrics에 요청(스크레이프)하여 정보를 조회한다. 본 수집기는 빠른 실행 속도와 호스트 컴퓨팅 자원의 오버헤드 최소화를 위해 Go 언어로 구현되었으며, 쿠버네티스 기반 인프라 환경과의 친화성을 위해 컨테이너 이미지로 빌드되어 파드(Pod) 형태로 배포된다.

#### 3.2 실험 설계

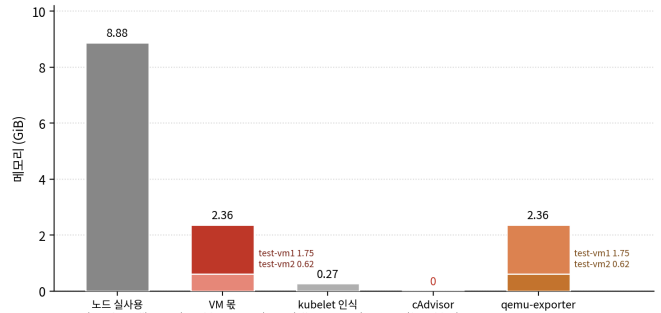
AWS EC2 클라우드 인스턴스 m5.2xlarge (8 vCPU, 32 GiB Memory) 환경에서 Ubuntu 24.04.4 LTS OS를 기반으로 실험을 구성하였다. OpenStack-Helm(OSH)을 활용하여 오픈스택의 제어부(Control Plane) 및 주요 구성 요소를 쿠버네티스 파드(Pod) 형태로 배포하였으며, 오픈스택을 통해 관측 대상 가상 머신(VM) 2대를 생성한

후 실측 실험을 진행하였다.

#### 3.3 실험 진행 및 결과

qemu-exporter의 필요성과 유효성을 검증하기 위해 세 가지 실험을 수행하였다. 첫 번째 실험에서는 VM의 메모리 사용량이 쿠버네티스 모니터링 체계에 누락되는지 여부를 kubelet과 cAdvisor를 통해 검증하였다.

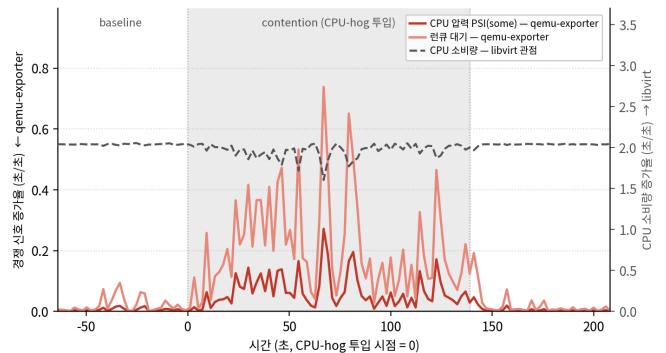
<그림 1> 메모리 사용량 인식 여부 : kubelet/cAdvisor와 qemu-exporter



실험 환경에서 VM의 메모리는 호스트 노드 실사용 메모리의 27%(2.36 GiB)를 점유하도록 배포 되었으나, kubelet이 인지한 메모리 요청량은 272 MiB에 불과했으며 cAdvisor 상에는 machine cgroup 관련 데이터가 존재하지 않았다. 반면, qemu-exporter는 Nova 메타데이터(인스턴스 ID, 이름, 인스턴스 타입 등)와 함께 각 VM의 메모리 사용량을 정확히 측정함을 확인하였다.

두 번째 실험에서는 CPU 자원 경쟁 상황을 유발하고, 이때 기존 libvirt 모니터링 그리고 제안한 qemu-exporter가 각각 경쟁을 측정할 수 있는지 비교 분석하였다. 부하 인가 전 60초, CPU 부하 인가 120초, 부하 종료 후 60초로 구성된 시나리오에서 2초 간격으로 각 도구의 CPU 사용률 및 PSI 데이터 등을 수집하였다.

<그림 2> CPU 자원 경쟁의 노출 여부 : libvirt와 qemu-exporter

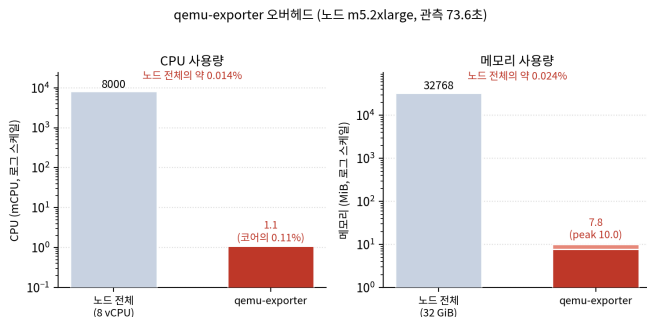


cgroup 내 스레드가 CPU를 할당받지 못한 시간 비율을 나타내는 PSI(some) 지표와 스레드별 대기 시간의 총합을 나타내는 schedstat(런큐 대기) 지표를 파싱하는 qemu-exporter는 부하 발생 시 CPU 미할당 정체 상태를 명확히 포착하였다. 구체적으로 부하 상황에서 PSI(some)

지표는 12.5배(0.0117 → 0.1467), 런큐 대기 지표는 10배(0.0454 → 0.4631) 상승하였다. 반면, libvirt 모니터링은 CPU 사용률 집계 시 대기 상태를 반영하지 않아[3], 부하가 지속됨에도 CPU 사용률이 3% 감소한 것으로만 측정되었다.

세 번째 실험에서는 qemu-exporter가 모니터링 에이전트로 동작할 때 호스트에 가하는 시스템 오버헤드를 측정하였다.

<그림 3> qemu-exporter가 호스트 컴퓨팅 자원을 차지하는 정도



측정 결과, 노드 전체에서 점유하는 CPU 사용량은 전체 코어의 0.11% 수준이었으며, 메모리 사용량은 피크(Peak) 시에도 10 MiB 수준에 그쳐 호스트 컴퓨팅 자원에 미치는 영향이 미미함을 확인하였다.

세 가지 실험 결과를 비교 요약하면 <표1>과 같다.

<표 1> libvirt, kubelet, qemu-exporter 각 모니터링 지표

| 관측 축                     | libvirt (virsh domstats) | cAdvisor/kubelet | qemu-exporter (제안) |
|--------------------------|--------------------------|------------------|--------------------|
| 자원 사용 (CPU, 메모리)         | ○                        | ×                | ○                  |
| 자원 경합 (압력, 대기)           | -                        | ×                | ○                  |
| Nova 정보 (uuid, flavor 등) | ○                        | ×                | ○                  |
| 노드 식별                    | ×                        | ○                | ○                  |

○ 제공 - 부분 제공 × 미제공

오픈스택의 기본 libvirt 기반 모니터링 방식은 CPU 및 메모리의 자원 사용률과 Nova 메타데이터를 제공한다. 그러나 자원 경합 정보의 경우 vcpu.N.wait 등 단편적인 누적 대기 시간만을 제공할 뿐 커널 수준의 PSI(Pressure Stall Information) 규격을 지원하지 않으며, 프로메테우스나 오픈텔레메트리(OpenTelemetry) 등 외부 모니터링 파이프라인 연동을 위한 노드(물리 서버) 식별 라벨 정보 또한 제공하지 않아 실시간 경합 상태를 관측하기 어렵다.

또한, 쿠버네티스 노드 내 파드 모니터링에 사용되는 cAdvisor 기반 kubelet은 VM이 관할 cgroup 영역에 속하지 않아 수집 자체가 불가능하다.

반면, 본 연구에서 제안한 qemu-exporter는 기존 libvirt 제공 정보에 더해 cgroup/procfs 파싱을 통한 자원 경합

지표를 통합 추출하고, 이를 외부 모니터링 도구 연동 규격으로 노출함을 검증하였다.

#### 4. 결론

본 연구는 OSH 환경에서 QEMU 프로세스가 쿠버네티스의 kubepods가 아닌 호스트의 machine cgroup 영역에 배치됨에 따른 모니터링 공백을 실측 검증하였다. cAdvisor 기반 kubelet은 VM 자원을 인지하지 못해 노드 메모리 사용률을 27% 과소평가하였으며, libvirt 기반 모니터링은 자원 경합 시 PSI(some)와 런큐 대기 지표가 각각 12.5배, 10배 폭증했음에도 CPU 사용률이 3% 감소에 그쳐 경합 상태를 효과적으로 포착하지 못했다.

이러한 '관측 경로의 공백'을 해소하고자 호스트 커널 데이터를 읽기 전용으로 수집하는 qemu-exporter를 제안하였다. 이를 통해 VM의 자원 사용률과 경합 지표를 누락 없이 수집하고, 적은 오버헤드로 VM과 OSH 구성 요소의 자원 상태를 단일 지점에서 통합 관측하는 실무적 기반을 마련하였다. 단, 본 실험은 AWS EC2 환경에서 KVM 가속 없이 순수 QEMU 에뮬레이션으로 수행되어 게스트 내부의 CPU Steal 지표를 확인하지 못한 한계가 있다. 향후 KVM 환경에서의 재현을 통해 게스트 관점의 경합 지표 포착 여부를 확인하는 것을 후속 과제로 남긴다.

#### 참고문헌

- [1] Cloud Native Computing Foundation, "Kubernetes Established as the De Facto 'Operating System' for AI as Production Use Hits 82% in 2025 CNCF Annual Cloud Native Survey," CNCF, 2026. (<https://www.cncf.io/announcements/2026/01/20/kubernetes-established-as-the-de-facto-operating-system-for-ai-as-production-use-hits-82-in-2025-cncf-annual-cloud-native-survey/>) (accessed Sep. 2026)
- [2] libvirt, "Control Groups Resource Management," libvirt.org, 2026. (<https://libvirt.org/cgroups.html>) (accessed Sep. 2026)
- [3] libvirt, "virsh(1) Manual — domstats," libvirt.org, 2026. (<https://libvirt.org/manpages/virsh.html#domstats>) (accessed Sep. 2026)
- [4] Kubernetes, "Tools for Monitoring Resources," Kubernetes Documentation, 2026. (<https://kubernetes.io/docs/concepts/cluster-administration/resource-usage-monitoring/>) (accessed Sep. 2026)